
DOCUMENTAZIONE PROGETTO D'ESAME

Equalizzatore Dinamico Multibanda con Analizzatore di Spettro FFT in Tempo Reale

Sommario

1. Obiettivi del progetto	2
2. File di Progetto	2
3. Considerazioni sulla GUI e rendering grafico.....	7
4. Guida all'utilizzo	13
5. Gruppo di Lavoro	14

1. Obiettivi del progetto

L'obiettivo fondamentale dell'architettura del nostro progetto è combinare un'equalizzazione dinamica sulle frequenze medie con una strumentazione di analisi spettrale e temporale integrata in un unico display grafico. A differenza di un equalizzatore statico tradizionale, in cui la risposta in frequenza rimane invariata a prescindere dall'ampiezza dell'ingresso, il sistema sviluppato misura l'ampiezza del segnale (involuppo) e rimodula dinamicamente il guadagno applicato alle frequenze centrali.

Parallelamente, l'architettura garantisce il monitoraggio visivo costante del segnale attraverso:

1. **Lo spettro FFT nel dominio delle frequenze.**
2. **L'oscilloscopio per la forma d'onda nel dominio del tempo.**
3. **La curva di risposta in frequenza dell'EQ, che riflette l'azione dei filtri.**

2. File di Progetto

- **PluginProcessor.h / PluginProcessor.cpp (Classe AudioPluginAudioProcessor)**
 - Rappresenta il nucleo DSP dell'applicazione. Riceve il buffer audio in ingresso, calcola l'involuppo del segnale nel tempo per rimodulare la dinamica della banda media, applica i filtri mediante le istanze della classe FiltroBiquad e inserisce i campioni processati all'interno di un buffer FIFO circolare per la visualizzazione.

PluginProcessor.cpp

```
// 3. Applica il filtraggio e l'effetto dinamico campione per campione
for (int i = 0; i < numeroCampioniBlocco; ++i)
{
    // Segue l'involuppo di volume del canale sinistro
    float ampiezzaAssoluta = std::abs(canaleSinistro[i]);
    involuppoLivelloMedi = 0.999f * involuppoLivelloMedi + 0.001f * ampiezzaAssoluta;
    float involuppo_dB = juce::Decibels::gainToDecibels(involuppoLivelloMedi + 1e-5f);

    // LOGICA DINAMICA: se il volume supera la soglia (threshold), abbassa i medi
    // automaticamente
    float gMediEffettivo = gMedi;
    if (involuppo_dB > sogliaDinamica_dB)
        gMediEffettivo -= (involuppo_dB - sogliaDinamica_dB) * 0.5f;

    // Imposta il filtro medi con il guadagno dinamico calcolato
    filtroMediL.impostaPeak((float)sampleRateDAW, fMedi, gMediEffettivo, qMedi);
}
```

```

filtroMediR.impostaPeak((float)sampleRateDAW, fMedi, gMediEffettivo, qMedi);
filtroMidRef = filtroMediL;

// --- FILTRAGGIO E SATURAZIONE SUL CANALE SINISTRO ---
float sL = filtroBassiL.elaboraCampione(canaleSinistro[i]);
sL = filtroMediL.elaboraCampione(sL);
sL = filtroAltiL.elaboraCampione(sL);
sL = std::tanh(sL * spintaDrive); // Applicazione saturazione valvolare (tangente
iperbolica)
canaleSinistro[i] = sL;

// --- FILTRAGGIO E SATURAZIONE SUL CANALE DESTRO ---
if (canaleDestro != nullptr)
{
    float sR = filtroBassiR.elaboraCampione(canaleDestro[i]);
    sR = filtroMediR.elaboraCampione(sR);
    sR = filtroAltiR.elaboraCampione(sR);
    sR = std::tanh(sR * spintaDrive);
    canaleDestro[i] = sR;
}

// --- TRASFERIMENTO DATI ALLA GRAFICA ---
if (indiceBufferFifo == dimensioneFFT)
{
    bloccoAudioProntoPerGUI = true;
    indiceBufferFifo = 0;
}
bufferFifoPerGUI[indiceBufferFifo++] = sL;
}
}

```

▪ **PluginEditor.h / PluginEditor.cpp (Classe AudioPluginAudioProcessorEditor)**

- Gestisce la componente visiva del plugin. Costruisce l'interfaccia utente, ospita i controlli di manipolazione (slider e manopole) ed eredita da **JUCE::Timer** per sincronizzare il refresh dell'interfaccia a circa 60 FPS senza bloccare o ritardare il thread audio principale.

PluginEditor.h (drawRotarySlider)

```

// Metodo speciale per disegnare la manopola circolare passo dopo passo
void drawRotarySlider(juce::Graphics& g, int x, int y, int width, int height,
                    float sliderPos, float rotaryStartAngle, float rotaryEndAngle,
                    juce::Slider& slider) override
{
    auto raggio = (float) juce::jmin(width, height) / 2.0f - 10.0f;
    auto centroX = (float) x + (float) width * 0.5f;
    auto centroY = (float) y + (float) height * 0.5f;
    auto rx = centroX - raggio;
    auto ry = centroY - raggio;
    auto angoloAccensione = rotaryStartAngle + sliderPos * (rotaryEndAngle - rotaryStartAngle);

    // Disegna il cerchio interno scuro
    g.setColour(juce::Colour(0xff222730));
    g.fillEllipse(rx, ry, raggio * 2.0f, raggio * 2.0f);
}

```

```

// Disegna il bordo scuro
g.setColour(juce::Colour(0xff12151B));
g.drawEllipse(rx, ry, raggio * 2.0f, raggio * 2.0f, 2.0f);

// Disegna l'arco azzurro luminoso indicatore del Livello
juce::Path arco;
arco.addCentredArc(centroX, centroY, raggio - 2.0f, raggio - 2.0f, 0.0f, rotaryStartAngle,
angoloAccensione, true);
g.setColour(slider.findColour(juce::Slider::rotarySliderFillColourId));
g.strokePath(arco, juce::PathStrokeType(3.0f));

// Disegna la lancetta bianca della manopola
juce::Path linea;
auto len = raggio * 0.65f;
linea.startNewSubPath(0.0f, -raggio);
linea.lineTo(0.0f, -raggio + len);
linea.applyTransform(juce::AffineTransform::rotation(angoloAccensione).translated(centroX,
centroY));
g.setColour(juce::Colours::white);
g.strokePath(linea, juce::PathStrokeType(2.5f));
}

```

▪ FiltroBiquad.h (Classe FiltroBiquad)

- Contiene il motore matematico per l'equalizzazione. Implementa le formule per il calcolo dei coefficienti dei filtri IIR di secondo ordine e applica la funzione di trasferimento al flusso dei campioni audio.

FiltroBiquad.h (elaboraCampione)

```

// Questa è la funzione che elabora un singolo campione di audio (un frammento infinitesimo di suono).
float elaboraCampione(float campioneIngresso)
{
    // Formula matematica (Direct Form II): combina l'ingresso attuale con il ricordo dei campioni passati
    float valoreIntermedio = campioneIngresso - coeffA1 * statoRitardo1 - coeffA2 * statoRitardo2;

    // Calcola il punto di suono finale applicando i coefficienti 'B'
    float campioneUscita = coeffB0 * valoreIntermedio + coeffB1 * statoRitardo1 + coeffB2 * statoRitardo2;

    // Fai scorrere la memoria indietro nel tempo per prepararti al prossimo campione
    statoRitardo2 = statoRitardo1; // Quello che era "ieri" diventa "l'altro ieri"
    statoRitardo1 = valoreIntermedio; // Il valore attuale diventa "ieri"

    return campioneUscita; // Restituisce il frammento di audio modificato
}

```

FiltroBiquad.h (impostaPeak)

```

// Imposta il filtro per lavorare sui MEDI (Filtro a Campana / Peak EQ)
void impostaPeak(float sampleRate, float frequenzaCentrale_Hz, float guadagno_dB, float fattoreQ)
{
    float A = std::pow(10.0f, guadagno_dB / 40.0f);
}

```

```

float omega0 = 2.0f * juce::MathConstants<float>::pi * frequenzaCentrale_Hz / sampleRate;
float cosw0 = std::cos(omega0);
// 'fattoreQ' regola quanto la campana d'azione del filtro deve essere stretta o larga
float alfa = std::sin(omega0) / (2.0f * juce::jmax(0.01f, fattoreQ));

float a0 = 1.0f + alfa / A;

coeffB0 = (1.0f + alfa * A) / a0;
coeffB1 = (-2.0f * cosw0) / a0;
coeffB2 = (1.0f - alfa * A) / a0;
coeffA1 = (-2.0f * cosw0) / a0;
coeffA2 = (1.0f - alfa / A) / a0;
}

```

FiltroBiquad.h (calcolaRispostaInFrequenza_dB)

```

// Calcola di quanti decibel (dB) il filtro aumenta o riduce una specifica frequenza (Hz).
// Serve per poter disegnare a schermo la curva azzurra dell'equalizzatore.
float calcolaRispostaInFrequenza_dB(float frequenzaTest_Hz, float sampleRateSistema) const
{
    // Converte la frequenza in Hz in radianti (omega)
    float omega = 2.0f * juce::MathConstants<float>::pi * frequenzaTest_Hz / sampleRateSistema;
    float cosw = std::cos(omega);
    float sinw = std::sin(omega);
    float cos2w = std::cos(2.0f * omega);
    float sin2w = std::sin(2.0f * omega);

    // Calcolo dei numeri complessi (parte reale e immaginaria) per numeratore e denominatore
    float reNum = coeffB0 + coeffB1 * cosw + coeffB2 * cos2w;
    float imNum = -coeffB1 * sinw - coeffB2 * sin2w;
    float reDen = 1.0f + coeffA1 * cosw + coeffA2 * cos2w;
    float imDen = -coeffA1 * sinw - coeffA2 * sin2w;

    // Calcola l'ampiezza (modulo)
    float modNum = std::sqrt(reNum * reNum + imNum * imNum);
    float modDen = std::sqrt(reDen * reDen + imDen * imDen);

    // Evita divisioni per zero ed esprime il risultato in Decibel (dB)
    float guadagnoLineare = modNum / juce::jmax(0.00001f, modDen);
    return juce::Decibels::gainToDecibels(guadagnoLineare);
}

```

▪ **VisualizzatoreSpettro.h / VisualizzatoreSpettro.cpp (Classe VisualizzatoreSpettro)**

- Componente custom d'interfaccia dedicato all'analisi visiva. Preleva i dati campionati dal buffer FIFO, applica l'algoritmo FFT per estrarre il modulo delle frequenze e disegna a schermo la combinazione dello Spettro FFT, dell'Oscilloscopio e della Curva di risposta in frequenza dell'EQ.

VisualizzatoreSpettro.cpp (riceviCampioniAudioPerFFT)

```

// Prende i campioni audio dell'engine e calcola la Trasformata di Fourier (FFT)
void VisualizzatoreSpettro::riceviCampioniAudioPerFFT(const float* bufferIngresso, int dimensioneBuffer)

```

```

{
    if (bufferIngresso == nullptr || dimensioneBuffer <= 0)
        return;

    int campioniDaCopiare = std::min(dimensioneBuffer, dimensioneFFT);

    // Salva L'onda audio per L'oscilloscopio
    std::memcpy(bufferWaveformLive, bufferIngresso, sizeof(float) *
static_cast<size_t>(campioniDaCopiare));

    // Trova L'ampiezza massima istantanea (per il pallino luminoso della saturazione)
    float maxAmp = 0.0f;
    for (int i = 0; i < campioniDaCopiare; ++i)
    {
        float absVal = std::abs(bufferIngresso[i]);
        if (absVal > maxAmp)
            maxAmp = absVal;
    }
    ampiezzaPiccoIstantaneo = maxAmp;

    juce::zeromem(campioniTempFFT, sizeof(campioniTempFFT));
    std::memcpy(campioniTempFFT, bufferIngresso, sizeof(float) *
static_cast<size_t>(campioniDaCopiare));

    // Applica La finestra di Hann e calcola Le frequenze con La FFT
    finestraDiHann.multiplyWithWindowingTable(campioniTempFFT, dimensioneFFT);
    calcolatoreFFT.performFrequencyOnlyForwardTransform(campioniTempFFT);

    // Converte Le ampiezze uscite dalla FFT in scala Decibel (dB) e Le scala da 0.0 a 1.0 per
il disegno
    for (int i = 0; i < dimensioneFFT / 2; ++i)
    {
        float livelloIn_dB = juce::Decibels::gainToDecibels(campioniTempFFT[i]) -
juce::Decibels::gainToDecibels(static_cast<float>(dimensioneFFT));
        float valoreNorm = juce::jmap(juce::jlimit(-70.0f, 0.0f, livelloIn_dB), -70.0f, 0.0f,
0.0f, 1.0f);

        livelloSpettroIstantaneo[i] = valoreNorm;

        // Peak Hold: tiene i picchi piú alti e li fa scendere piano piano
        if (valoreNorm > tracciaPicchiMassimi[i])
            tracciaPicchiMassimi[i] = valoreNorm;
        else
            tracciaPicchiMassimi[i] *= 0.98f;
    }
}
}

```

▪ **GestoreFileAudio.h / GestoreFileAudio.cpp (Classe GestoreFileAudio)**

- Modulo dedicato alla gestione dell'I/O audio da disco. Fornisce la logica per aprire un **juce::FileChooser**, decodificare il file audio selezionato e inviarlo al flusso di elaborazione del filtro.

GestoreFileAudio.cpp (leggiCampioneConInterpolazione)

```

// Ritorna il valore del campione corrente. Se la scheda audio va a una velocità diversa,
// calcola il punto preciso tramite interpolazione lineare (media tra due campioni vicini).

```

```

float GestoreFileAudio::leggiCampioneConInterpolazione(int canale, double sampleRateDAW)
{
    juce::ignoreUnused(sampleRateDAW);

    // Se siamo in stop o non c'è audio, restituisce silenzio (0)
    if (!staRiproducendo || bufferTracciaAudio.getNumSamples() == 0)
        return 0.0f;

    int totaleCampioni = bufferTracciaAudio.getNumSamples();
    int idx1 = (int) indiceLetturaCampione; // Primo punto vicino
    int idx2 = (idx1 + 1) % totaleCampioni; // Secondo punto vicino
    float alpha = (float) (indiceLetturaCampione - idx1); // Distanza decimale tra i due punti

    int ch = canale % bufferTracciaAudio.getNumChannels();
    float y1 = bufferTracciaAudio.getSample(ch, idx1);
    float y2 = bufferTracciaAudio.getSample(ch, idx2);

    // Formula dell'interpolazione lineare
    return y1 + alpha * (y2 - y1);
}

```

3. Considerazioni sulla GUI e rendering grafico

L'interfaccia utente grafica (GUI) del plugin è sviluppata all'interno della classe **AudioPluginAudioProcessorEditor** ed è progettata per offrire un controllo visivo immediato e rigoroso sia del segnale audio in ingresso/uscita, sia della risposta spettrale e temporale introdotta dalla sezione di filtraggio dinamico.

Per rispettare i vincoli di un'elaborazione in tempo reale (*real-time audio processing*), la GUI è completamente disaccoppiata dal thread audio principale (**AudioPluginAudioProcessor**): l'interfaccia visiva opera nel thread grafico principale (*main GUI thread*), evitando qualsiasi potenziale blocco (*audio drop-out* o *glitch*) dovuto a operazioni pesanti di disegno o calcolo analitico.

Componente Principale: VisualizzatoreSpettro

L'elemento centrale della GUI è il componente personalizzato **VisualizzatoreSpettro** (definito nei file **VisualizzatoreSpettro.h** e **VisualizzatoreSpettro.cpp**), il quale gestisce un rendering visivo composito suddiviso in tre strati sovrapposti:

1. Spettro FFT (Dominio delle Frequenze):

Rappresenta l'energia frequenziale del segnale audio distribuita nell'intervallo udibile da 20 Hz a 20 kHz su scala logaritmica. Il calcolo viene effettuato tramite la classe **juce::dsp::FFT** impostata su una dimensione di blocco $N = 2048$ campioni (2^{11}). Prima dell'esecuzione della trasformata, ai campioni accumulati viene applicata una

finestra di Hann per minimizzare la dispersione spettrale (*spectral leakage*). I valori di ampiezza risultanti vengono convertiti in scala logaritmica espressa in decibel (dB).

VisualizzatoreSpettro.cpp (disegnaSpettroFFTLive)

```
void VisualizzatoreSpettro::disegnaSpettroFFTLive(juce::Graphics& g, juce::Rectangle<float> area)
{
    auto plot = disegnaGrigliaEAssi(g, area, "2. SPETTRO FFT LIVE & PEAK HOLD", "dB");

    juce::Graphics::ScopedSaveState saveState(g);
    g.reduceClipRegion(plot.toNearestInt());

    juce::Path tracciaFFT, tracciaPeak;
    bool primo = true;
    int larghezzaPixel = static_cast<int>(plot.getWidth());

    for (int x = 0; x < larghezzaPixel; ++x)
    {
        float normX = static_cast<float>(x) / static_cast<float>(larghezzaPixel);
        // Scala Logaritmica delle frequenze da 20 Hz a 20.000 Hz
        float freqHz = 20.0f * std::pow(20000.0f / 20.0f, normX);

        int binIdx = static_cast<int>((freqHz / (sampleRateRiferimento / 2.0f)) * (dimensioneFFT / 2));
        binIdx = juce::jlimit(0, (dimensioneFFT / 2) - 1, binIdx);

        float valFFT = livelloSpettroIstantaneo[binIdx];
        float valPeak = tracciaPicchiMassimi[binIdx];

        float xPos = plot.getX() + static_cast<float>(x);
        float yPos = juce::jlimit(plot.getY(), plot.getBottom(), plot.getBottom() - (valFFT * plot.getHeight()));
        float yPeakPos = juce::jlimit(plot.getY(), plot.getBottom(), plot.getBottom() - (valPeak * plot.getHeight()));

        if (primo)
        {
            tracciaFFT.startNewSubPath(xPos, yPos);
            tracciaPeak.startNewSubPath(xPos, yPeakPos);
            primo = false;
        }
        else
        {
            tracciaFFT.lineTo(xPos, yPos);
            tracciaPeak.lineTo(xPos, yPeakPos);
        }
    }

    // Linea rosa trasparente dei picchi
    g.setColour(juce::Colour(0x88FF4081));
    g.strokePath(tracciaPeak, juce::PathStrokeType(1.0f));

    // Spettro istantaneo viola
    g.setColour(juce::Colour(0xffE040FB));
    g.strokePath(tracciaFFT, juce::PathStrokeType(1.6f));
}
```

2. Oscilloscopio (Dominio del Tempo):

Traccia la forma d'onda del segnale audio in tempo reale, consentendo di apprezzare la variazione di ampiezza dei picchi e l'impatto dell'involuppo dinamico sulla risposta temporale del segnale.

VisualizzatoreSpettro.cpp (disegnaOscilloscopioLive)

```
// 1. DISENI DELL'OSCILLOSCOPIO (Linea Blu)
void VisualizzatoreSpettro::disegnaOscilloscopioLive(juce::Graphics& g, juce::Rectangle<float> area)
{
    auto plot = disegnaGrigliaEAssi(g, area, "1. OSCILLOSCOPIO LIVE (Tempo)", "Ampl");

    // Limita rigidamente il disegno dentro la griglia
    juce::Graphics::ScopedSaveState saveState(g);
    g.reduceClipRegion(plot.toNearestInt());

    juce::Path traccia;
    bool primo = true;
    int larghezzaPixel = static_cast<int>(plot.getWidth());

    for (int x = 0; x < larghezzaPixel; ++x)
    {
        float normX = static_cast<float>(x) / static_cast<float>(larghezzaPixel);
        int sampleIdx = static_cast<int>(normX * (dimensioneFFT - 1));
        float val = bufferWaveformLive[sampleIdx];

        float xPos = plot.getX() + static_cast<float>(x);
        float rawY = plot.getY() + juce::jmap(val, -1.0f, 1.0f, 1.0f, 0.0f) * plot.getHeight();
        float yPos = juce::jlimit(plot.getY(), plot.getBottom(), rawY);

        if (primo) { traccia.startNewSubPath(xPos, yPos); primo = false; }
        else { traccia.lineTo(xPos, yPos); }
    }

    g.setColour(juce::Colour(0xff2962FF));
    g.strokePath(traccia, juce::PathStrokeType(1.6f));
}
```

3. Curva di Risposta in Frequenza dell'EQ:

Mostra il profilo di attenuazione/esaltazione calcolato teoricamente a partire dai coefficienti Correnti dei filtri IIR Biquad. La curva viene aggiornata dinamicamente per riflettere le variazioni di guadagno introdotte dall'azione dell'involuppo sulle frequenze medie.

VisualizzatoreSpettro.cpp (disegnaCurvaEQLive)

```
// 3. DISEGNO DELLA CURVA DELL'EQUALIZZATORE (Linea Azzurra)
void VisualizzatoreSpettro::disegnaCurvaEQLive(juce::Graphics& g, juce::Rectangle<float> area)
{
    auto plot = disegnaGrigliaEAssi(g, area, "3. RISPOSTA EQ TEORICA  $|H(e^{j\omega})|$ ", "dB");
}
```

```

juce::Graphics::ScopedSaveState saveState(g);
g.reduceClipRegion(plot.toNearestInt());

juce::Path tracciaEQ;
bool primo = true;
int larghezzaPixel = static_cast<int>(plot.getWidth());

for (int x = 0; x < larghezzaPixel; ++x)
{
    float normX = static_cast<float>(x) / static_cast<float>(larghezzaPixel);
    float freqHz = 20.0f * std::pow(20000.0f / 20.0f, normX);

    // Somma la risposta in frequenza dei 3 filtri (Bassi + Medi + Alti)
    float guadagno_dB = filtroLowRif.calcolaRispostaInFrequenza_dB(freqHz,
sampleRateRiferimento)
        + filtroMidRif.calcolaRispostaInFrequenza_dB(freqHz,
sampleRateRiferimento)
        + filtroHighRif.calcolaRispostaInFrequenza_dB(freqHz,
sampleRateRiferimento);

    float xPos = plot.getX() + static_cast<float>(x);
    float rawY = plot.getCentreY() - (guadagno_dB / 24.0f) * (plot.getHeight() * 0.5f);
    float yPos = juce::jlimit(plot.getY(), plot.getBottom(), rawY);

    if (primo) { tracciaEQ.startNewSubPath(xPos, yPos); primo = false; }
    else { tracciaEQ.lineTo(xPos, yPos); }
}

g.setColour(juce::Colour(0xff00E5FF));
g.strokePath(tracciaEQ, juce::PathStrokeType(2.0f));
}

```

Comunicazione Thread-Safe e Sincronizzazione a 60 FPS

La comunicazione tra il processore audio e l'interfaccia grafica avviene in modo *thread-safe* mediante una struttura dati a **buffer circolare FIFO (First-In-First-Out)**.

- **Scrittura nella FIFO:** Durante l'esecuzione di **processBlock** nel file **PluginProcessor.cpp**, i campioni d'uscita elaborati vengono memorizzati in tempo reale all'interno del buffer FIFO.

PluginProcessor.cpp (scrittura bufferFifoPerGUI)

```

// --- TRASFERIMENTO DATI ALLA GRAFICA ---
if (indiceBufferFifo == dimensioneFFT)
{
    bloccoAudioProntoPerGUI = true;
    indiceBufferFifo = 0;
}
bufferFifoPerGUI[indiceBufferFifo++] = sL;
}

```

- **Lettura e Refresh Grafico:** La classe **AudioPluginAudioProcessorEditor** eredita da **juce::Timer** ed esegue la callback temporizzata **timerCallback()** con un intervallo di circa 16.6 ms, garantendo un aggiornamento fluido a **60 Frame Per Secondo (FPS)**. Quando la FIFO contiene un numero sufficiente di campioni ($N = 2048$), la classe **VisualizzatoreSpettro** estrae i dati, applica la finestra di Hann, esegue l'FFT e ordina il ridisegno visivo via **repaint()**.

PluginEditor.cpp (timerCallback)

```
// Timer a 60Hz per inviare i dati audio alla grafica
void AudioPluginAudioProcessorEditor::timerCallback()
{
    if (audioProcessor.bloccoAudioProntoPerGUI)
    {
        float valoreDriveAttuale =
audioProcessor.alberoParametriPlugin.getRawParameterValue("drive")->load();

        visualizzatoreSpettro.riceviCampioniAudioPerFFT(audioProcessor.bufferFifoPerGUI,
AudioPluginAudioProcessor::dimensioneFFT);
        visualizzatoreSpettro.impostaFiltriDiRiferimento(
            audioProcessor.filtroLowRef,
            audioProcessor.filtroMidRef,
            audioProcessor.filtroHighRef,
            48000.0f,
            valoreDriveAttuale
        );
        audioProcessor.bloccoAudioProntoPerGUI = false;
    }
}
```

Flusso operativo audio e DSP

Il flusso di elaborazione del segnale all'interno del plugin segue una sequenza rigorosa e deterministica che collega il caricamento da disco alla modifica spettrale e alla conseguente rappresentazione grafica.

PluginProcessor.cpp (processBlock)

```
// QUESTA È LA FUNZIONE PRINCIPALE CHE ELABORA L'AUDIO AD OGNI ISTANTE
void AudioPluginAudioProcessor::processBlock (juce::AudioBuffer<float>& bufferAudio,
juce::MidiBuffer&)
{
    juce::ScopedNoDenormals noDenormals;
    bufferAudio.clear(); // Pulisce il contenitore dell'audio

    // Se siamo in STOP, non fa nulla
    if (!gestoreFileAudio.isRiproduzioneAttiva())
        return;

    int numeroCampioniBlocco = bufferAudio.getNumSamples();
    int numeroCanaliAudio = bufferAudio.getNumChannels();

    // 1. Legge l'audio dal file e lo inserisce nel buffer
    for (int s = 0; s < numeroCampioniBlocco; ++s)
```

```

{
    for (int ch = 0; ch < numeroCanaliAudio; ++ch)
    {
        float campioneLetto = gestoreFileAudio.leggiCampioneConInterpolazione(ch,
sampleRateDAW);
        bufferAudio.setSample(ch, s, campioneLetto);
    }
    gestoreFileAudio.avanzaPosizioneLettura(sampleRateDAW);
}

// 2. Legge i valori attuali di tutte le manopole impostate dall'utente
float gBassi = alberoParametriPlugin.getRawParameterValue("lowGain")->load();
float fBassi = alberoParametriPlugin.getRawParameterValue("lowFreq")->load();

float gMedi = alberoParametriPlugin.getRawParameterValue("midGain")->load();
float fMedi = alberoParametriPlugin.getRawParameterValue("midFreq")->load();
float qMedi = alberoParametriPlugin.getRawParameterValue("midQ")->load();

float gAlti = alberoParametriPlugin.getRawParameterValue("highGain")->load();
float fAlti = alberoParametriPlugin.getRawParameterValue("highFreq")->load();

float spintaDrive = alberoParametriPlugin.getRawParameterValue("drive")->load();
float sogliaDinamica_dB = alberoParametriPlugin.getRawParameterValue("threshold")->load();

// Aggiorna i filtri dei bassi e degli alti
filtroBassiL.impostaLowShelf((float)sampleRateDAW, fBassi, gBassi);
filtroBassiR.impostaLowShelf((float)sampleRateDAW, fBassi, gBassi);

filtroAltiL.impostaHighShelf((float)sampleRateDAW, fAlti, gAlti);
filtroAltiR.impostaHighShelf((float)sampleRateDAW, fAlti, gAlti);

// Mantiene una copia aggiornata per permettere al grafico di tracciare la linea EQ
filtroLowRef = filtroBassiL;
filtroHighRef = filtroAltiL;

auto* canaleSinistro = bufferAudio.getWriter(0);
auto* canaleDestro = bufferAudio.getNumChannels() > 1 ? bufferAudio.getWriter(1) :
nullptr;

// 3. Applica il filtraggio e l'effetto dinamico campione per campione
for (int i = 0; i < numeroCampioniBlocco; ++i)
{
    // Segue l'involuppo di volume del canale sinistro
    float ampiezzaAssoluta = std::abs(canaleSinistro[i]);
    involuppoLivelloMedi = 0.999f * involuppoLivelloMedi + 0.001f * ampiezzaAssoluta;
    float involuppo_dB = juce::Decibels::gainToDecibels(involuppoLivelloMedi + 1e-5f);

    // LOGICA DINAMICA: se il volume supera la soglia (threshold), abbassa i medi
    automaticamente
    float gMediEffettivo = gMedi;
    if (involuppo_dB > sogliaDinamica_dB)
        gMediEffettivo -= (involuppo_dB - sogliaDinamica_dB) * 0.5f;

    // Imposta il filtro medi con il guadagno dinamico calcolato
    filtroMediL.impostaPeak((float)sampleRateDAW, fMedi, gMediEffettivo, qMedi);
    filtroMediR.impostaPeak((float)sampleRateDAW, fMedi, gMediEffettivo, qMedi);
    filtroMidRef = filtroMediL;

    // --- FILTRAGGIO E SATURAZIONE SUL CANALE SINISTRO ---

```

```

float sL = filtroBassiL.elaboraCampione(canaleSinistro[i]);
sL = filtroMediL.elaboraCampione(sL);
sL = filtroAltiL.elaboraCampione(sL);
sL = std::tanh(sL * spintaDrive); // Applicazione saturazione valvolare (tangente
iperbolica)
canaleSinistro[i] = sL;

// --- FILTRAGGIO E SATURAZIONE SUL CANALE DESTRO ---
if (canaleDestro != nullptr)
{
    float sR = filtroBassiR.elaboraCampione(canaleDestro[i]);
    sR = filtroMediR.elaboraCampione(sR);
    sR = filtroAltiR.elaboraCampione(sR);
    sR = std::tanh(sR * spintaDrive);
    canaleDestro[i] = sR;
}

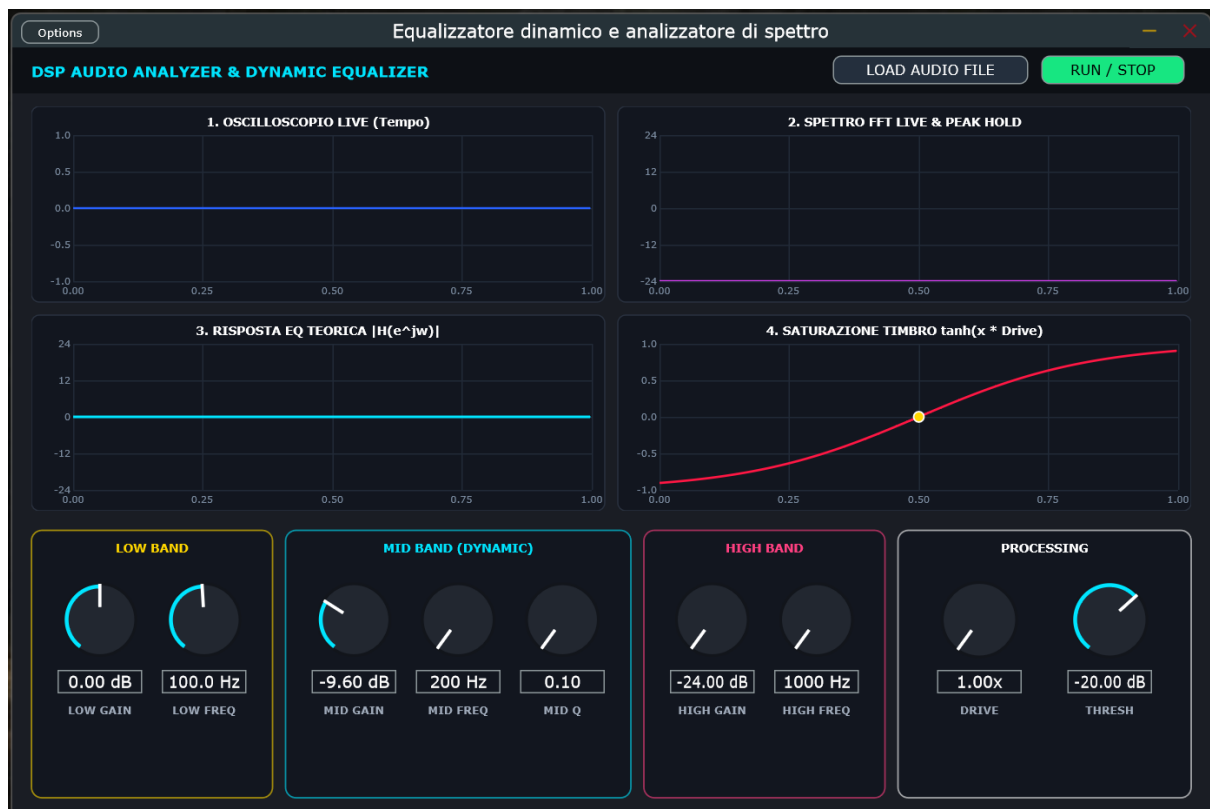
// --- TRASFERIMENTO DATI ALLA GRAFICA ---
if (indiceBufferFifo == dimensioneFFT)
{
    bloccoAudioProntoPerGUI = true;
    indiceBufferFifo = 0;
}
bufferFifoPerGUI[indiceBufferFifo++] = sL;
}
}

```

4. Guida all'utilizzo

1. **Caricamento Traccia Audio:** Fare clic sull'apposito pulsante nell'interfaccia grafico per selezionare un file audio (.wav/.mp3) tramite la finestra **juce::FileChooser**.
2. **Regolazione Equalizzazione Base:** Modificare la frequenza centrale f_c , il fattore Q e il guadagno base fisso tramite i potenziometri della GUI.
3. **Impostazione della Dinamica dei Medi:** Regolare la soglia (*Threshold*) e i tempi di attacco/rilascio dell'involuppo per osservare l'adattamento dinamico del guadagno sui segnali ad ampiezza variabile.

4. **Monitoraggio Grafico:** Osservare sul display superiore l'evoluzione sovrapposta dello spettro audio (FFT), dell'onda temporale e della risposta dei filtri.



5. Gruppo di Lavoro

Ai fini della valutazione del progetto, l'attività di sviluppo è stata ripartita in modo equilibrato secondo la struttura modulare del framework JUCE:

- **Amalia Elena Oprea (s1109394) — Architettura DSP e Motore Audio:**
 - Progettazione e implementazione della classe **FiltroBiquad** e calcolo dei coefficienti matematici.
 - Sviluppo dell'algoritmo di **Envelope Follower** e della modulazione dinamica dei medi all'interno di **PluginProcessor.cpp**.
 - Implementazione della classe **GestoreFileAudio** per l'I/O audio da disco rigido.
- **Santiago Politi (s1109217) — Architettura Grafica, FFT e GUI:**

- Sviluppo della classe **VisualizzatoreSpettro**, calcolo dell'FFT con finestra di Hann e rendering delle tre curve visive.
- Progettazione della GUI visiva e dei controlli parametrici in **PluginEditor.cpp**.
- Gestione del timer di sincronizzazione a 60 FPS e implementazione thread-safe della memoria FIFO per lo scambio dati audio-grafica.